Семинар 5. Репликация, модели согласованности и CAP-теорема

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Сдвинут DL по первому домашнему заданию – теперь в 23:59 21.03.2026
- Второе домашнее задание – жесткий дедлайн в 23:59 02.04.2026
  - Полностью с автопроверкой (10 баллов за галочку в GitHub Classroom)
  - Штраф за сдачу после дедлайна - 50%

Содержание

1. Пессимистическая репликация
2. Оптимистическая репликация
3. CAP- и PACELC- теоремы
4. Модели согласованности

Понятие репликации

Репликация

Избыточное дублирование одного логического объекта на нескольких узлах системы.

- Зачем нужна:
  - **Доступность**: не нужно далеко ходить за данными (данные всегда доступны, даже если другие узлы лежат)
  - **Производительность**: параллелизм на чтение, распределение нагрузки
  - **Георасределение**: доступность в плане географических зон
- Цена репликации:
  - необходимость управлять целостностью данных (integrity)
  - конфликты и неопределенный порядок событий

Две стратегии репликации

Стратегия 1: Пессимистическая (синхронная)

Перед тем как ответить клиенту на запись, синхронизируем со всеми репликами.

- Плюс: гарантия что все видят одно и то же
- Минус: запись медленна, если упадет какой-то узел — система не отвечает
- Разрешение конфликтов a priori

Стратегия 2: Оптимистическая (асинхронная)

Клиенту ответили сразу, дальше все реплики синхронизируются в фоне.

- Плюс: запись быстра, система отвечает даже если узлы падают
- Минус: временно разные реплики видят разные значения
- Разрешение конфликтов a posteriori

Пессимистическая репликация

Пессимистическая репликация

Синхронная репликация: все изменения синхронизируются **до** ответа клиенту.

- Почему она называется «пессимистической»?
  - Инварианты не могут быть временно нарушены
  - Баланс счета не должен уйти в минус даже временно
  - Логин/email должны быть уникальны *сейчас*
  - Ограничения внешних ключей
- Как работает пессимистическая репликация:
  - Координатор (мастер) синхронизирует запись со всеми репликами
  - Используется quorum/consensus: большинство должны подтвердить
  - Только когда большинство согласны, клиент получает ответ
- Типичные механизмы:
  - Distributed locks (мьютекствуло между узлами)
  - Consensus/quorum (Raft, Paxos)
  - Two-Phase Commit (2PC)

Транзакционная модель ACID

То, какие данные и с какой целью реплицируются, – это детали реализации системы. Внешняя модель согласованности, свойственная пессимистическим транзакциям, – это ACID:

- **Atomicity**: транзакция либо целиком применена, либо откат
- **Consistency**: переход между корректными состояниями БД
- **Isolation**: параллельные транзакции не мешают и не заметны друг другу
- **Durability**: после commit данные переживают сбой узла (WAL)

Важно

ACID — это свойства *модели выполнения транзакций*, а не «strong consistency». Но сам подход является пессимистическим.

Linearizability и One-Copy Serializability (1SR)

- **Linearizability**:
  - каждая операция выглядит как мгновенная в момент между invoke/response
  - если A завершилась раньше, чем B началась, то A должна быть раньше B в глобальном порядке
  - модель «поведения одной корректной копии в реальном времени»
- **One-Copy Serializability (1SR)**:
  - итог эквивалентен некоторому последовательному порядку транзакций над одной копией
  - не требует сохранения реального времени между независимыми транзакциями

Комментарий

Linearizability – для атомарных операций записи/чтения,
1SR-Serializability – для транзакций.

Блокировка

Понятие блокировки

Механизм, ограничивающий параллельный доступ к данным для обеспечения их согласованности:

- **Shared (S)** — читать можно многим, писать не может никто
- **Exclusive (X)** — читать и писать может только один

- Централизованный координатор блокировок
  - **coordinator lock service** (например, ZooKeeper/etcd)
  - модель check-in/check-out
  - single point of failure, но может децентрализованным с помощью consensus
- Two-Phase Locking (2PL):
  - growing phase: только захватываем lock'и
  - shrinking phase: только освобождаем
- Риски: deadlocks, high contention (дисциплина обслуживания)

Распределенная блокировка

Понятие распределенной блокировки

Блокировка на некоторое время (TTL = Time-to-Live) между узлами в распределенной системе.

Fencing token

Монотонно растущий номер владения lock'ом; хранилище обязано отклонять устаревшие токены.

- **lease-based lock** с TTL
- Практика: lock + idempotency + timeout + retry с backoff

Rate Limiter с помощью распределенной блокировки

Задача: ограничить количество запросов одного пользователя до N в минуту.

Наивный подход с глобальной блокировкой

Для каждого пользователя есть счетчик `cnt[user]`. При каждом запросе:

1. захватить `lock[user]` (прежде чем трогать счетчик)
2. прочитать `cnt[user]`
3. если `cnt[user] < N`: инкрементировать, разрешить запрос
4. иначе: отклонить (HTTP 429 Too Many Requests)
5. освободить `lock[user]`

Optimistic Concurrency Control (OCC)

Позволяем тразакциям конкурировать, но проверяем их на конфликт в момент commit. Если конфликт — отклоняем одну из транзакций.

Пример сценария

Сотрудник Alice и сотрудник Bob редактируют зарплату одного человека.

- Alice читает: `salary = 50000`, версия `v=5`
- Bob читает: `salary = 50000`, версия `v=5`
- Alice пишет: update to 55000 где `v=5`
- Bob пишет: update to 52000 где `v=5`

Из этих двух обновлений только одно может быть принято.

Увлекательный аспект

Техника называется оптимистической, потому что предполагает, что конфликты редки и не стоит блокировать ресурсы заранее. Но она пессимистическая, потому что в момент commit мы pessimistically проверяем, что никто не изменил данные с момента чтения.

Содержание

1. Пессимистическая репликация
2. Оптимистическая репликация
3. CAP- и PACELC- теоремы
4. Модели согласованности

Оптимистическая репликация

Оптимистическая репликация

Репликация, при которой мы оптимистически разрешаем write-обновления на репликах без предварительной синхронизации.

- Как это работает:
  - У каждого узла системы есть своя реплика данных
  - Чтение и запись производятся с локальной реплики, без синхронизации
  - Состояния синхронизируются синхронно (gossip, anti-entropy)
- Как работать с конфликатми:
  - Детерминированно их разрешать без синхронизации (CRDT)
  - Полуавтоматически и централизованно (семантическая реконсиляция)
  - Полностью ручной централизованной merge

Транзакционная модель BASE

- **Basically Available**: данные доступны для чтения и записи на узле даже при сбоях других узлов
- **Soft state**: состояние всегда спекулятивное, т.е. мы читаем не «последнее», а «какое-то» значение
- **Eventually consistent**: при отсутствии новых записей реплики сходятся

ACID vs BASE

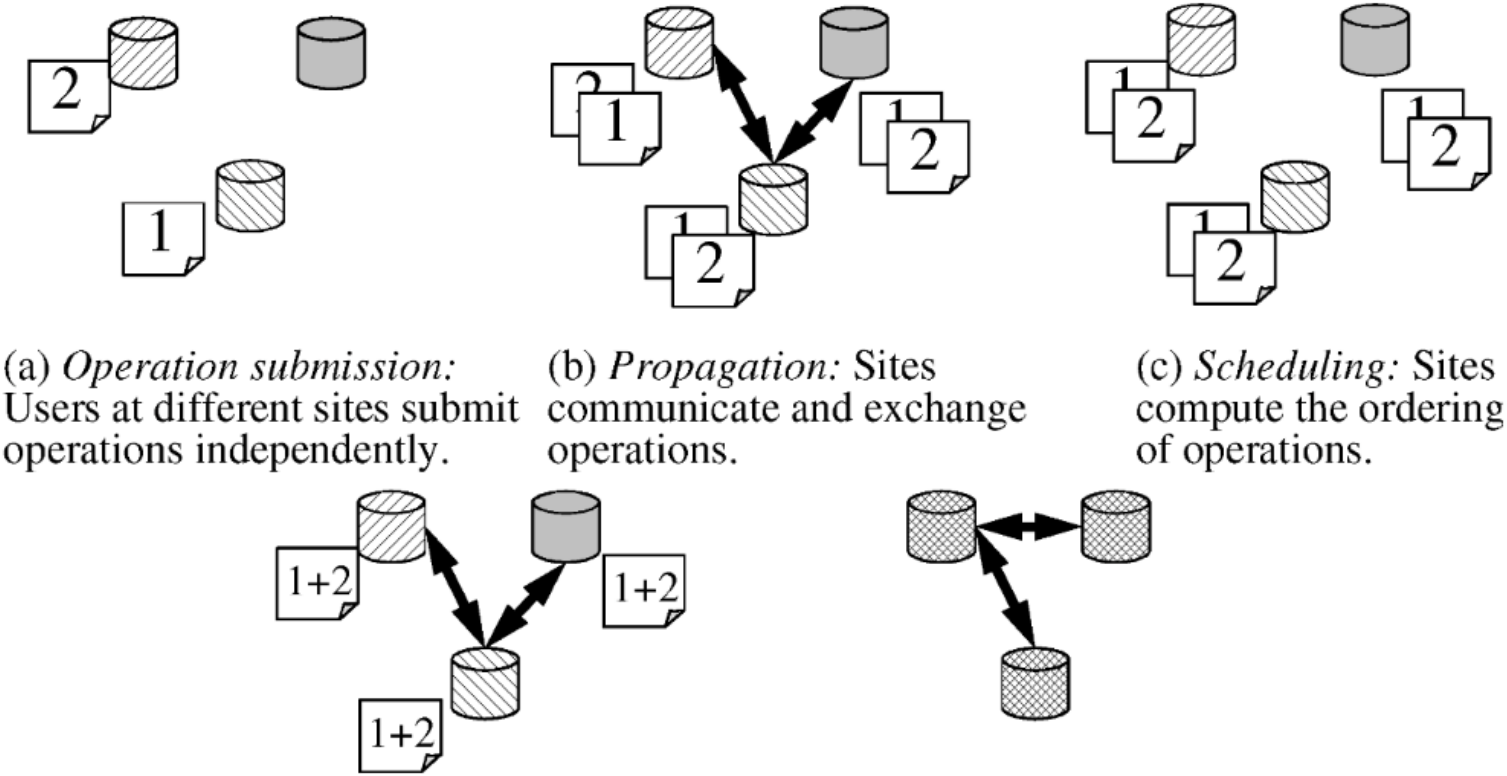
ACID оптимизирует корректность синхронного commit, BASE — доступность и масштаб в условиях partition.

Статья 'Optimistic Replication': M. Shapiro et al. (2005)

- Ввела общий язык для optimistic replication и reconciliation
- Ключевые оси классификации:
  - state-transfer vs operation-transfer
  - синхронная vs асинхронная доставка
  - автоматический vs ручной merge
- Главный вывод: нельзя «бесплатно» получить сразу простоту, низкую задержку и сильную согласованность
- Практика: проектировать **типы данных и операции** под возможные конфликты заранее

Схема оптимистической репликации

Y. Saito and M. Shapir



(a) *Operation submission:* Users at different sites submit operations independently.

(b) *Propagation:* Sites communicate and exchange operations.

(c) *Scheduling:* Sites compute the ordering of operations.

(d) *Conflict resolution:* Sites detect conflicts and transform offending operations to produce results intended by users.

(e) *Commitment:* Sites agree on the final ordering and reconciliation result. Their changes become permanent.

Fig. 1. Elements of optimistic replication and their roles. Disks represent replicas, memo sheets represent operations, and arrows represent commu-

Знакомые до боли векторные часики

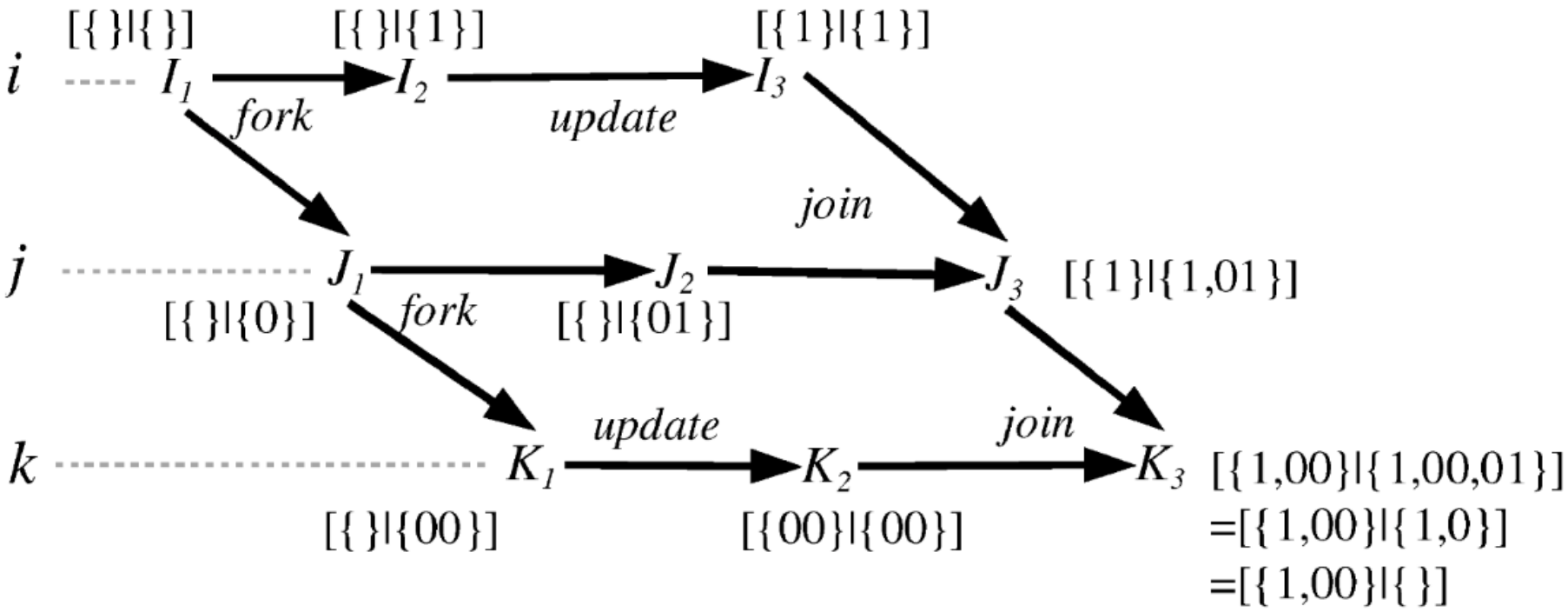


Fig. 13. Example of the use of version timestamps (VTs). An object starts as a single replica I_1 with a VT of [{}|{}]. It is forked into two replicas I_2 and J_1. Site i updates the replica, which becomes I_3. Merging replicas I_3 and J_2 detects no conflict as I_3 dominates J_2, as apparent from the fact that $\{1\} \supset \{\}$. In contrast, concurrent updates are detected when merging replicas J_3 and K_2 as neither of the upd-ids, {00} and {1}, subsumes the other.

Модель Amazon DynamoDB и прикол с корзиной покупок

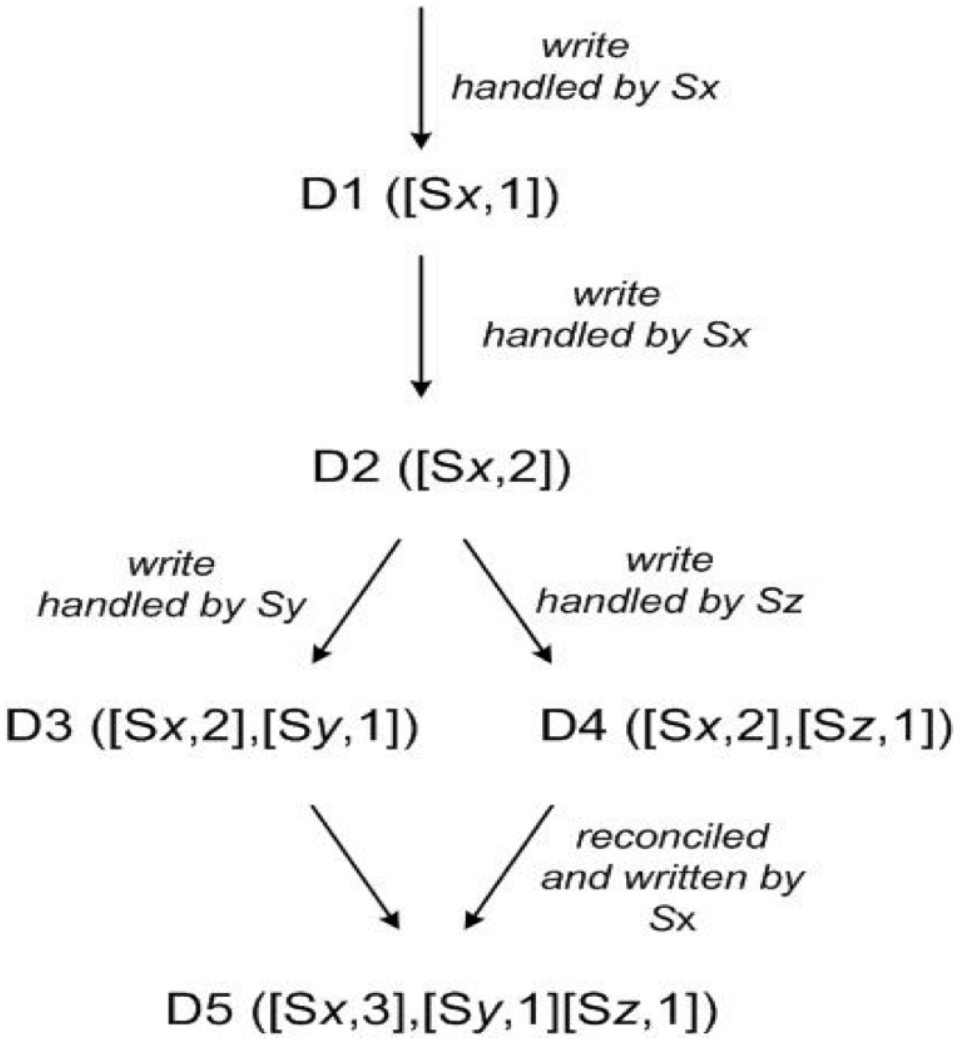


Figure 3: Version evolution of an object over time.

Содержание

1. Пессимистическая репликация
2. Оптимистическая репликация
3. CAP- и PACELC- теоремы
4. Модели согласованности

Почему две стратегии, а не одна?

- Ранее мы увидели два **противоположных** подхода к репликации
- Пессимистический: жертвуем доступностью и задержкой за консистентность
- Оптимистический: жертвуем консистентностью за доступность и задержку

Три невозможных желания

Нельзя одновременно гарантировать:

- **C**onsistency: все видят одно и то же
- **A**vailability: система всегда отвечает
- **P**artition tolerance: система работает когда сеть разломана

Необходимые понятия

Network partition — когда узлы перестают иметь возможность коммуницировать

- **Consistency (C)**: все клиенты видят единое согласованное состояние
- **Availability (A)**: каждый корректный запрос получает ответ
- **Partition tolerance (P)**: система продолжает работу при потере связности между узлами
- **Latency (L)** — время ответа на запрос

Деталь

В CAP под C обычно понимают поведение, близкое к linearizability.

CAP-теорема

CAP-теорема (Brewer, 2000; Gilbert & Lynch, 2002)

В системе нельзя одновременно гарантировать (max 2/3):

- **C**onsistency: сильная согласованность данных
- **A**vailability: доступность
- **P**artition tolerance: устойчивость к сетевым разрывам

Детали про CAP-теорему

- **CP**: отклоняем часть запросов, но не отдаем противоречивые ответы
- **AP**: отвечаем всегда, но допускаем временную рассогласованность

Классы систем

Любая система относится либо к CP (пессимистическая), либо к AP (оптимистическая).

Важный момент

CA-система возможна только если игнорировать partition как класс сбоев, что наивно.

'Critique of CAP theorem' (Kleppmann, 2014)

- Сетевые разрывы – это настолько очевидный тип отказа, что глупо говорить отдельно, что мы можем его игнорировать (P)
- По факту выбираем между C и A, при этом упускаем измерение L (latency)
- Подход Клеппманна:
  - Разделим операции на delay-dependent и delay-independent
  - Для delay-independent операций можно достичь и C, и A
  - Иначе выбираем C или A в зависимости от требований к задержке

PACELC-теорема

- **If P** (partition): выбор между **A** и **C**
- **Else** (нет partition): выбор между **L** (latency) и **C**
- Примеры:
  - **PA/EL**: Dynamo-подобные системы (в норме низкая задержка)
  - **PC/EC**: системы, жертвующие latency ради строгой согласованности

Инженерный смысл

Компромисс между скоростью и консистентностью существует всегда, даже когда сеть «здоровая».

Содержание

1. Пессимистическая репликация
2. Оптимистическая репликация
3. CAP- и PACELC- теоремы
4. Модели согласованности

Понятие согласованности

- **Модель согласованности** задает, какие значения при чтении система может вернуть после записей
- Это контракт между клиентским кодом и хранилищем
- Чем слабее модель, тем проще масштабироваться и выше доступность
- Чем сильнее модель, тем меньше аномалий, но выше задержка

Consistency vs. Integrity

Consistency (видимость)

- Какие допустимые состояния системы видны нам после изменения
- Должны ли мы видеть обновление? С какой допустимой задержкой? В каком порядке?
- Пример: read-your-writes (session consistency)

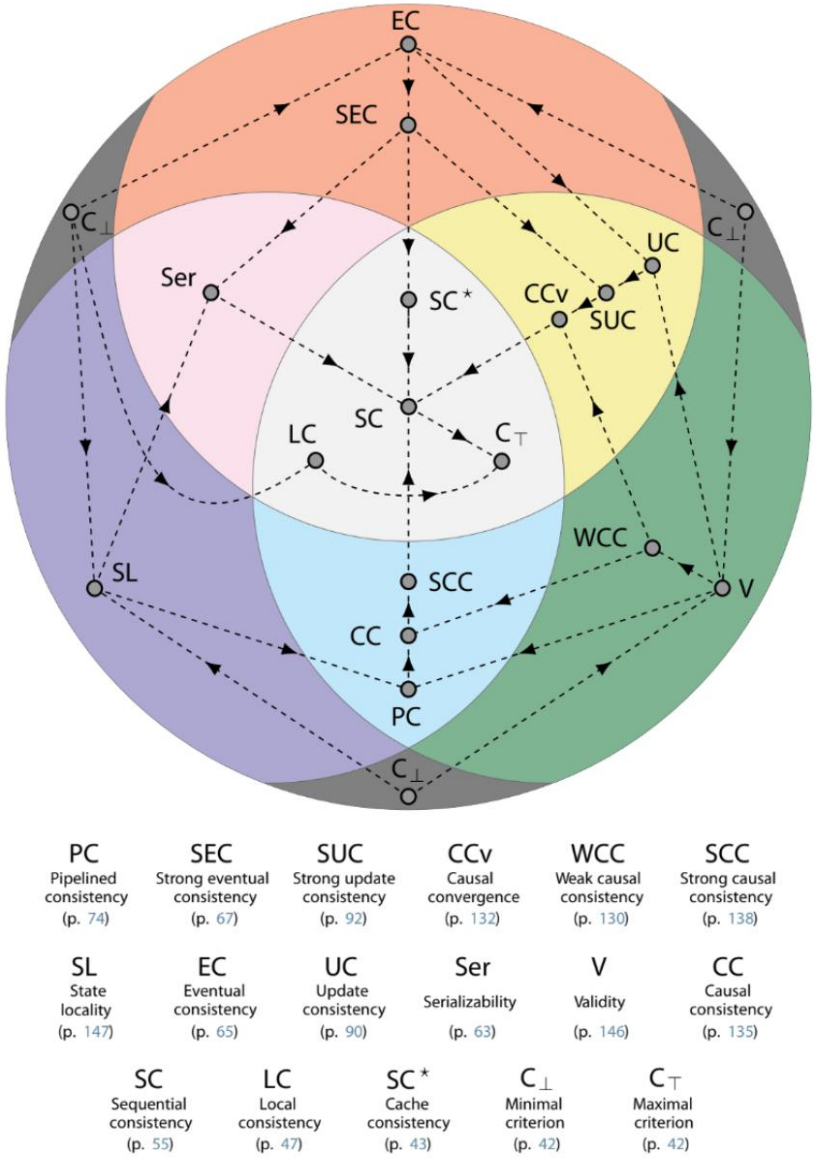
Integrity (инварианты)

- Какие состояния запрещены
- Пример: `balance >= 0`, уникальность email

Это два параллельных измерения

Можно иметь strong consistency и при этом нарушать integrity из-за неверной бизнес-логики.

Некоторые важные модели по M. Perrin



1. **Strong Consistency**
   Все операции читают последнее записанное значение
2. **Sequential Consistency**
   Существует общий линейный порядок операций
3. **Causal Consistency**
   Happens-before порядок сохраняется
4. **Eventual Consistency**
   Спустя какое-то время реплики сойдутся
5. **Strong Eventual Consistency**
   Реплика = множество полученных ей сообщений

Strong Consistency

- Любое чтение возвращает результат последней завершенной записи
- Поведение эквивалентно «одной копии» данных
- Плюсы: простой mental model для разработчика
- Минусы: выше latency (часто нужен кворум/лидер), чувствительность к partition

Подходит: платежи, инвентарь, дедупликация критичных операций.

Sequential Consistency

- Все операции наблюдаются в одном и том же **глобальном порядке**
- Этот порядок согласован с **порядком операций каждого отдельного клиента**
- Но не обязан совпадать с реальным временем

Следствие

Операция, завершившаяся раньше по wall-clock, может оказаться позже в глобальном порядке.

Strong vs. Sequential Consistency

Свойство	**Strong**	**Sequential**
Единый порядок операций	Да	Да
Учет реального времени	Да	Нет
Влияние сетевых задержек	Выше	Ниже

Грубо говоря: strong = sequential + real-time constraints.

Семейство моделей Weak Consistency

- Не гарантируют единого «актуального» значения в каждый момент
- Часто дают локальные гарантии:
  - Read Your Writes
  - Monotonic Reads
  - Monotonic Writes
  - Writes Follow Reads
- Идея: выбрать **минимально достаточную** гарантию под конкретный use-case

Eventual Consistency

- Если новые записи прекратились, все реплики в итоге сходятся
- Не гарантирует срок сходимости
- На практике требуется:
  - анти-энтропия
  - детерминированный merge
  - контроль clock skew (для LWW)

Типичный сценарий: кеши, каталоги, ленты рекомендаций.

Causal Consistency

- Сохраняет отношение happens-before между причинно связанными операциями
- Конкурентные (независимые) записи могут наблюдаться в разном порядке
- Пример:
  - пользователь A публикует пост
  - пользователь B отвечает на пост
  - нельзя увидеть ответ, не увидев исходный пост

Механизмы: vector clocks, dependency tracking.

Strong Eventual Consistency

- При доставке одного и того же набора обновлений все реплики гарантированно равны
- Порядок доставки не важен
- Обычно достигается через CRDT:
  - операции коммутативны
  - merge ассоциативен, коммутативен и идемпотентен

Примеры: счетчики, множества, совместное редактирование документов.

Session Consistency

- Гарантии только внутри одной клиентской сессии
- Наиболее полезные свойства:
  - Read Your Writes
  - Monotonic Reads
- После реконнекта к другому региону гарантия может временно пропасть
- Практика: sticky sessions или session token с версией

Как выбирать модель согласованности на практике

1. Выписать бизнес-инварианты (что **никогда** нельзя нарушить)
2. Разделить операции на критичные и некритичные
3. Назначить минимальную достаточную модель на операцию
   - платеж/списание: strong или serializable транзакции
   - лента/счетчики просмотров: eventual или causal
   - профиль пользователя: session guarantees часто достаточно
4. Контролировать production через p95/p99 latency и частоту конфликтов

Итоги

- (!) Выдано второе домашнее задание
- Обсудили понятие репликации и две противоположные стратегии: пессимистическую и оптимистическую
- CAP-теорема иллюстрирует, почему важны и нужны оба подхода
- Разные модели согласованности – разные гарантии видимости данных